

1)Apache Airflow'un Temel Bileşenleri.

Airflow bileşenleri açıklamaları:

DAG: Projedeki işlerin hangi sırayla, birbirine bağlı olarak nasıl çalışacağını gösteren iş haritasıdır.

Scheduler: Arka planda sürekli nöbet tutan ve zamanı gelen işleri sıraya alıp çalıştıran zamanlayıcıdır.

Executor: Sıradaki işlerin bilgisayarda hangi yöntemle ve nasıl koşturulacağına karar veren mekanizmadır.

Worker: Sıraya alınan işleri üstlenip bilgisayarda fiilen çalıştıran işçi birimlerdir.

Metadata Database: İşlerin durumunu, geçmişini ve tüm Airflow ayarlarını hafızada tutan veri tabanıdır.

Web UI: İşlerin durumunu renkli grafiklerle izlediğimiz, hata olunca loglara baktığımız yönetim panelidir.

2)Kurduğum Mimari Kararlar ve İş Akışı Açıklaması:

İş Akışı Sıralaması: Projede BashOperator kullanarak 3 temel görevi sıraya dizdim. İlk olarak trigger_spark_ingestion göreviyle ham verileri Spark üzerinden veri tabanına aldım. Bu işlem başarıyla tamamlandığı an, dbt dönüşümlerini başlatan trigger_dbt_transformation görevi devreye giriyor. En son adımda ise analitik panoların güncellenmesini simüle eden bir yapı kurguladım.

Operatör Seçimi Kararı: Projede ağır veri yüklerini çalıştırmak için BashOperator tercih ettim. Bunun nedeni, Airflow'un bilgisayarı ve sistemi yormasını engellemek, onu sadece süreçleri yöneten bir "beyin" olarak konumlandırmaktır. Ağır hesaplama ve dönüşüm yükleri doğrudan kendi motorlarına (Apache Spark ve dbt) bırakılmıştır. Bu karar sistem kaynaklarını korumak ve çökmeleri engellemek için alınmıştır.

3)Medallion Mimarisi ve Modüler Dönüşümler:

Projede dbt kullanarak verileri 3 farklı katmana ayırdım ve modüler hale getirdim:

Bronz Katmanı (Ham Veri): src_olist.yml dosyası ile veri tabanındaki işlenmemiş ham Olist tablolarını sisteme kaynak olarak tanıttım.

Gümüş Katmanı (Temizlik): stg_olist_orders.sql ve stg_olist_customers.sql dosyalarında veriyi temizledim. Tarih formatlarını düzelttim, metinlerdeki gereksiz boşlukları sildim ve veri tiplerini standartlaştırdım.

Altın Katmanı (Yıldız Şeması): fact_orders.sql dosyasında, dbt'nin ref() fonksiyonunu kullanarak Gümüş katmanındaki temiz tabloları bağladım. Böylece analiz araçlarının doğrudan okuyabileceği, nihai ve tertemiz bir Yıldız Şema (Fact) Tablosu oluşturdum.

Kurduğumuz bu mimariyi gösteren diyagram sizin bize ödevde verdiğiniz diyagram.

Bu Mimarinin Faydaları:

Hata Bulmak Kolaylaştı (Modülerlik): Veriyi tek bir devasa kodla dönüştürmek yerine parçalara (modüllere) böldüm. Bir hata olduğunda Bronz'da mı, Gümüş'te mi yoksa Altın'da mı olduğunu anında görebiliyorum.

Veri Kalitesi Arttı: Temizleme işlemlerini Gümüş katmanında bitirdiğim için, Altın katmanındaki raporlama tabloya hiçbir zaman bozuk veya eksik veri sızıyor.

Yeniden Kullanılabilirlik: Gümüş katmanında hazırladığım temiz müşteri tablosunu, ileride başka bir analiz için tekrar sıfırdan temizlik yapmadan doğrudan kullanabilirim.

4)Yeniden Yapılandırma Sürecindeki Zorlu Kısımlar

Ortam ve Yol (PATH) Uyuşmazlıkları: Kendi bilgisayarımda Windows ve Anaconda sanal ortamı kullanırken dbt araçlarının terminal komut yollarını (PATH) ayarlamak ve kütüphanelerin birbiriyle çakışmasını engellemek kurulum aşamasında beni en çok zorlayan kısımlardan biriydi. Teknik araçların birbiriyle senkronize çalışması için ortam değişkenlerini manuel olarak yapılandırmam gerekti.

Mükerrer Verinin Engellenmesi (Idempotency): Sistemi üst üste veya farklı zamanlarda birkaç kez çalıştırdığımda, veri tabanında aynı verilerin tekrar tekrar kopyalanarak mükerrer (duplicate) kayıtlar oluşturmasını engellemek mimari açıdan zorlayıcıydı. dbt katmanında dönüşümleri tasarlarken sistemin her çalışmada eski verinin üstüne yazmasını veya tutarlı, tekil sonuçlar üretmesini sağlamak üzerine kafa yormam gerekti.